

SAGA

Pattern

Mülakat Hazırlık Dokümanı

Mikroservis mimarilerinde distributed transaction yönetimi

Tanım • 2PC karşılaştırması • Compensating transaction
Orchestration ve choreography • Isolation problemi
Gerçek hayat kullanımı • Mülakat cevap kalıpları

İçindekiler

1. Giriş ve Temel Tanım
2. Saga Neden Gerekli? — Database-per-service, Scaling, 2PC
3. Compensating Transaction
4. Orchestration vs Choreography
5. Orchestration Mekanizması — Detaylı
6. Isolation Problemi ve Çözümleri
7. Gerçek Hayatta Saga Kullanımı
8. Mülakat Hazırlık — Sorular, Cevaplar, Cheat Sheet
9. Kapanış ve Akılda Kalsın Hook'ları

1. Giriş ve Temel Tanım

Saga Pattern Nedir?

Saga pattern, mikroservis mimarilerinde birden fazla servisi kapsayan iş süreçlerinin tutarlılığını sağlamak için kullanılan bir patterndir. Klasik ACID transaction'ları kullanamadığımız durumlarda — yani her servisin kendi veritabanına sahip olduğu database-per-service prensibinde — bir iş akışını **birbirini takip eden lokal transaction'lar zinciri** olarak modelleriz. Herhangi bir adım başarısız olursa, daha önce başarıyla tamamlanmış adımları geri almak için **compensating transaction'lar** çalıştırılır. Sistem bu şekilde **eventual consistency** ile tutarlı duruma getirilir.

Saga'nın iki uygulama yaklaşımı vardır: **orchestration** (merkezi bir koordinatör akışı yönetir) ve **choreography** (servisler birbirlerine event'lerle haberleşir). İkisinin de avantajları ve riskleri farklıdır; hangisinin tercih edileceği akışın karmaşıklığına ve servis sayısına bağlıdır.

Mülakatta Söylenecek Tek Cümle

Saga pattern, mikroservis mimarilerinde birden fazla servisi kapsayan iş süreçlerinin tutarlılığını sağlayan bir patterndir. ACID transaction'ı kullanamadığımız durumlarda bir iş akışını lokal transaction'lar zincirine böler, hata durumunda daha önce tamamlanmış adımları compensating transaction'larla mantıksal olarak geri alır ve sistem eventual consistency ile tutarlı duruma getirilir. İki yaklaşımı vardır: orchestration ve choreography.

Tarihsel Arka Plan

Saga konsepti 1987 yılında Hector Garcia-Molina ve Kenneth Salem'in "Sagas" başlıklı paper'ında ortaya atıldı. O zamanlar problem uzun süren transaction'ların (long-lived transactions) veritabanı kaynaklarını çok uzun süre kilitlemesiydi. Çözüm: işlemi alt-transaction'lara böl, her biri kendi commit'ini yapsın, hata olursa compensating action'larla geri al. Mikroservis dünyası bu konsepti yeniden keşfetti ve günümüzün dağıtık sistemlerinde transactional consistency sağlamak için standart yaklaşımlardan biri haline geldi.

2. Saga Neden Gerekli?

Database-per-Service Prensibi

Mikroservis mimarisinin temel taşlarından biridir: **her mikroservisin kendine ait bir veritabanı vardır ve başka servisler bu veritabanına direkt erişemez.** Veriye yalnızca o servisin API'sı üzerinden ulaşılır.

Neden böyle olmalı:

- **Schema coupling olmaması:** A servisi tabloya kolon eklediğinde B etkilenmemeli, bağımsız deploy edilebilmeli
- **Performans izolasyonu:** A servisinin heavy query'si B'yi yavaşlatmamalı
- **Teknoloji özgürlüğü:** A servisi MongoDB, B PostgreSQL kullanabilmeli
- **Implicit coupling olmaması:** Servisler API kontratı üzerinden bağlanmalı, schema üzerinden değil

Paylaşılan veritabanı kullanmak, görünüşte mikroservis ama gerçekte **distributed monolith** sonucunu doğurur. Database-per-service bu tuzağı önler.

Bedeli: Bir iş akışı birden fazla servisin veritabanına yazacaksa, tek bir global ACID transaction kuramazsın. İşte Saga tam bu noktada doğar.

Scaling — "Ölçeklenmek" Ne Demek?

Scaling, sistemin yük arttıkça (kullanıcı, istek, veri) performansını koruyabilme yeteneğidir. İki türü vardır.

Vertical scaling (scale-up)

Tek bir makineyi güçlendirmek. Örneğin 8 GB RAM'i 64 GB RAM'e çıkarmak. Limiti vardır, pahalıdır, ve makine çöktüğünde tüm sistem çöker (single point of failure).

Horizontal scaling (scale-out)

Makine sayısını artırmak. Örneğin 10 sunucuyu 100 sunucuya çıkarmak. Modern bulut sistemler bu yaklaşım üzerine kuruludur. Pratikte sınırsızdır, daha ucuzdur, fault-tolerant'tır.

"**Ölçeklenmiyor**" demek, yük arttıkça performansın katastrofik şekilde bozulması ya da artırılabilmesi demektir. Bir sistemin yatay ölçeklenebilmesi için: paylaşılan state minimum olmalı, kilitler az tutulmalı ve sistem mümkün olduğunca asenkron çalışmalı. Her shared lock, her synchronous bekleyen call, her merkezi koordinasyon noktası bir bottleneck oluşturur.

Neden 2PC (Two-Phase Commit) Kullanmıyoruz?

2PC, distributed transaction için klasik protokoldür. İki fazda çalışır:

- **Prepare phase:** Coordinator tüm participant'lara "hazır mısınız?" sorar. Her participant kaynakları kilitler ve "hazırım" cevabı verir.
- **Commit phase:** Hepsi evet derse coordinator "commit" der, biri hayır derse "abort" der.

Mikroservislerde 2PC neden ölçeklenmez:

Sorun	Açıklama
Synchronous blocking	Prepare boyunca kaynaklar kilitli, throughput dramatik düşer
Multiplicative failure	5 servisin %99.9 uptime'ı %99.5'e iner (yıllık 43h downtime). Hepsi aynı anda ayakta olmalı.
Coordinator SPOF	Coordinator prepare ile commit arasında çökerse participantlar belirsizlikte kalır, kilitler açılmaz
Coordinator bottleneck	Tüm transaction'lar tek coordinator'dan geçer, yatay ölçeklenmez
Network round-trip maliyeti	5 servis × 2 phase = 10+ round-trip, latency birikiyor
DB desteği yetersiz	NoSQL'lerin ve modern DB'lerin çoğu XA protokolünü desteklemiyor
CAP teoreminde availability'den feda	Network partition'da sistem durur — mikroservislerde genelde availability daha kritik
Tight coupling	Tüm servisler aynı anda ayakta ve coordinator'a bağlı olmalı — mikroservis felsefesinin tersi

Black Friday Senaryosu — Somut Örnek

Saniyede 10.000 sipariş geliyor. Her sipariş 5 servisi etkiliyor: Order, Payment, Inventory, Shipping, Notification. 2PC ile bir transaction kaynakları yaklaşık 200 ms kilitliyorsa, aynı popüler ürünün stok satırına **saniyede maksimum 5 sipariş** işlenebilir. Bin kişi aynı ürüne saldırınca 999'u kuyrukta bekler, sistem cevap veremez hale gelir.

Saga karşılığında ne sunuyor: Asenkron çalışma, kilit kullanmama, eventual consistency, yatay ölçeklenebilirlik. Servisler bağımsız çalışır, biri çökse diğerleri devam eder, aynı kaynağa paralel binlerce istek akabilir.

3. Compensating Transaction

Tanım — Rollback Değildir

Compensating transaction, zaten commit edilmiş bir lokal transaction'ın etkisini **mantıksal olarak tersine çeviren YENİ bir transaction'dır**. Database rollback DEĞİLDİR — yeni bir forward operation'dır.

Bu nüansı mülakatta mutlaka vurgula: Saga'da "rollback" yoktur. Çünkü her servis kendi lokal transaction'ını **zaten commit etmiştir**, veriyi kalıcı hale getirmiştir. Veritabanı seviyesinde rollback yapamazsın. Onun yerine, yapılan işlemi mantıksal olarak nötrleyen yeni bir işlem çalıştırırsın.

Örnekler

Forward Action (yapılan iş)	Compensating Transaction (telafi)
Ödeme alındı	Refund (iade) işlemi yapılır
Stoktan düşüldü	Stok geri eklenir
Rezervasyon yapıldı	Rezervasyon iptal edilir
Bilet oluşturuldu	Bilet "cancelled" statüsüne çekilir
Mail gönderildi	"İptal edildi" maili gönderilir (semantic rollback)
Hesaba para yatırıldı	Aynı tutar geri çekilir, yeni bir transaction olarak

Olmazsa Olmaz Özellikler

1. Idempotent olmalı

Aynı compensating transaction birden fazla kez çalışsa bile sonuç aynı olmalı. Çünkü retry mekanizmaları çalışır, network sorunları olur. İki kez refund tetiklenirse iki kez para iade ETMEMELİ. Bu genelde unique transaction ID + database constraint kombinasyonu ile sağlanır.

2. Eventually successful olmalı

Compensating transaction çalışmazsa sistem tutarsız kalır. Bu yüzden retry mekanizması, dead letter queue ve manuel müdahale alarmları gerekir. Sonunda mutlaka başarılı olmalı.

3. Her forward action için tanımlı olmalı

Saga tasarımında her adım için önceden compensating action'ı belirlersin. Sonradan "acaba nasıl geri alırız" düşünmek tasarım hatasıdır.

4. Semantic rollback gerektirebilir

Geri alınamayan işlemler için bile mantıksal tersine çevirme yapılır. Mail gönderildi mi? Maili "silemezsiniz", ama "iptal edildi" maili göndererek etkisini nötrleyebilirsiniz. Bu yaklaşıma semantic rollback denir.

4. Orchestration vs Choreography

Saga'nın iki uygulama yaklaşımı vardır. Hangisinin tercih edileceği akışın karmaşıklığına, servis sayısına ve organizasyonun teknik tercihinə bağlıdır.

Choreography — "Dans Grubu"

Merkez yok. Servisler event-driven çalışır. Bir servis işini tamamlayınca event yayınlar; ilgilenen servisler bu event'i dinleyip kendi işlerini yaparlar. Publish/subscribe modelidir, mesaj kuyrukları üzerinden (Kafka, RabbitMQ vb.) çalışır.

Örnek akış:

- | | |
|----------------------|---|
| 1. Order Service | → "OrderCreated" event yayar |
| 2. Payment Service | → "OrderCreated" dinler, ödeme alır
→ "PaymentCompleted" event yayar |
| 3. Inventory Service | → "PaymentCompleted" dinler, stoğu düşer
→ "StockReserved" event yayar |
| 4. Shipping Service | → "StockReserved" dinler, kargo açar
→ "ShipmentCreated" event yayar |

Orchestration — "Orkestra Şefi"

Merkezi bir orchestrator bütün akışı yönetir. Servisleri sırayla çağırır, cevaplarını bekler, sonraki adıma geçer. Bir hata olduğunda compensating transaction'ları yine orchestrator tetikler.

Pseudo-code örneği:

```
OrderOrchestrator.execute(order):  
  step 1: payment.processPayment(...)  
    if fail: return CANCELLED  
  step 2: inventory.reserveStock(...)  
    if fail: payment.refund()  
    return CANCELLED  
  step 3: shipping.createShipment(...)  
    if fail: inventory.releaseStock()  
    payment.refund()  
    return CANCELLED  
  step 4: notification.send(...)  
    (non-critical, fail loglanır, akış devam eder)  
  return SUCCESS
```

7 Adımlı E-Commerce Senaryosu — Somut Karşılaştırma

Düşün: bir sipariş akışı 7 adımdan oluşsun — order create, payment, stok düş, kargo, bildirim, loyalty puan, analytics.

Choreography ile yaz — bir sorun çıkar: stok yoksa ne olacak? Inventory Service "StockFailed" event'i yayınlar. Birisi bunu dinleyip ödemeyi iade etmek zorunda. Bu Payment Service'tir. Yani Payment Service artık şunları bilmek zorunda:

- **OrderCreated** dinler → ödeme al
- **StockFailed** dinler → iade et

- **ShipmentFailed** dinler → iade et
- **FraudDetected** dinler → iade et
- **AdminCancelledOrder** dinler → iade et
- **WarehouseError** dinler → iade et

Payment Service "ben sadece para alıp veriyorum" servisi olmaktan çıktı, sistemin yarısının event'lerine boğuldu. Aynı sorun Order Service için de geçerli: siparişi cancel etmek için PaymentFailed, StockFailed, ShipmentFailed, FraudDetected... hepsini dinlemek zorunda.

Distributed Monolith Tuzağı

Yukarıdaki durum **distributed monolith** anti-pattern'idir. Görünüşte bağımsız mikroservisler vardır ama gerçekte: her servis başka servislerin event'lerine bağımlıdır, yeni bir adım eklemek zincirleme değişikliklere yol açar, compensation cascade'i kontrolsüzdür, debug çok zorlaşır. Mikroservisin getirdiği avantajlar — bağımsız deploy, loose coupling — bu durumda kaybolur.

Karşılaştırma Tablosu

Özellik	Orchestration	Choreography
Kontrol	Merkezi orchestrator	Yok, event-driven
Haberleşme	Direkt komut/cevap	Pub/sub events
Akış görünürlüğü	Tek yerde, kolay	Dağınık, tracing şart
Coupling	Daha sıkı (orchestrator her servisi bilir)	Daha gevşek görünür ama servis bazında her servis başkalarının event'lerini bilir
Debug kolaylığı	Yüksek	Düşük
Risk	Orchestrator SPOF olabilir	Cyclic dependency, distributed monolith
Compensation yönetimi	Sırası belli, kolay	Cascading event yönetimi zor
Conditional logic	Merkezi, basit	Her serviste, dağınık
Uygun olduğu yer	4+ servis, karmaşık akış, finansal işlemler	2-3 servis, basit doğrusal akış

Hangisi Ne Zaman?

Choreography iyi seçimdir eğer:

- Akış 2-3 servislilik ve doğrusal

- Gerçek event-driven mimari mevcut (Kafka, herkes event düşünüyor)
- Servislerin gerçekten bağımsız evolve etmesi gerekiyor

Orchestration iyi seçimdir eğer:

- 4 ve daha fazla servis var
- Karmaşık conditional logic var ("premium kullanıcı ise X, değilse Y")
- Sıkı compensation gereken finansal işlemler
- Visibility ve auditing kritik (regülasyon)
- Akış sık değişebiliyor

5. Orchestration Mekanizması — Detaylı

Yaygın Yanlış Anlama: "Orchestrator failure'ı nasıl öğreniyor?"

Mülakatta ve günlük tartışmalarda en sık karıştırılan nokta budur. Cevap şu: **orchestrator failure'ı bir event yayınlamasından değil, çağırdığı servisten gelen HTTP cevabından öğrenir.**

Servisler "ben başarısız oldum" diye event publish etmek zorunda DEĞİL — sadece kendilerini çağıran orchestrator'a fail return etmeleri yeterli.

Orchestrator pasif değildir, AKTİFTİR. Telefon eden o, servisler cevap veren. Choreography'de servisler pasif yayıncı, orchestrator yok — kim ilgileniyorsa kendi dinler. Orchestration'da orchestrator aktif çağırıcı, servisler cevap veren.

Mobil Sipariş Senaryosu — Adım Adım Trace

Kullanıcı mobilden sipariş veriyor, ödeme adımında kartında para yok. Orchestration ile bu akış nasıl yürüyor — kod seviyesinde adım adım izle.

Adım 1: Mobil HTTP isteği gönderir

```
POST https://api.shop.com/orders
Body: {
  userId: 123,
  productId: 456,
  qty: 2,
  paymentMethodId: 789
}
```

Adım 2: Controller orchestrator'ı çağırır

```
@RestController
public class OrderController {

    @Autowired private OrderOrchestrator orchestrator;

    @PostMapping("/orders")
    public ResponseEntity<OrderResult> placeOrder(
        @RequestBody Order order) {
        OrderResult result = orchestrator.execute(order);
        return ResponseEntity.ok(result);
    }
}
```

Adım 3: Orchestrator satır satır çalışır

```
@Service
public class OrderOrchestrator {

    @Autowired private InventoryClient inventoryClient;
    @Autowired private PaymentClient paymentClient;

    public OrderResult execute(Order order) {

        // SATIR 1: Stok kontrolü
        // -> HTTP GET inventory-service/check
```

```

// -> Inventory Service kendi DB'sinde sorgu atar
// -> Cevap: {available: true}
// Orchestrator bu cevaba kadar BLOKE OLUR.
StockResponse stockResp = inventoryClient.checkStock(
    order.productId, order.qty);
if (!stockResp.available) {
    return OrderResult.fail("STOK_YOK");
}

// SATIR 2: Stok rezerve et
// -> HTTP POST inventory-service/reserve
// -> Stok 2 azalir, reservation_id doner
ReserveResponse reserveResp = inventoryClient.reserveStock(
    order.productId, order.qty);

// SATIR 3: Karttan parayi cek <-- KRITIK SATIR
// -> HTTP POST payment-service/charge
// -> Payment Service banka API'sini cagirir
// -> BANKA: "yetersiz bakiye"
// -> Payment Service cevap doner:
// {success: false, reason: "INSUFFICIENT_FUNDS"}
PaymentResponse payResp = paymentClient.charge(
    order.userId, order.total, order.paymentMethodId);

if (!payResp.success) {
    // ORCHESTRATOR ARTIK BILİYOR: KART YETERSİZ
    // Cunku Payment Service ona success=false dondurdu.
    // Hicbir event yayilmadi, hickimse hicbir sey dinlemiyor.
    // Sadece bir HTTP cevabi geldi.

    // Compensation: rezerve edilmiş stogu geri sal
    inventoryClient.releaseStock(reserveResp.reservationId);

    return OrderResult.fail("ODEME_BASARISIZ: " + payResp.reason);
}

// ... akisin devamı
return OrderResult.success(...);
}
}

```

Anahtar zihinsel model: Orchestrator'daki *paymentClient.charge(...)* satırı arka planda HTTP isteği atıyor ve cevap dönene kadar Java thread'ini BLOKE EDİYOR. Cevap geldiğinde değişkene atanıyor. Bu, herhangi bir REST API çağrısından farklı değil. "Haberdar olma" diye ayrı bir mekanizma yok — CEVAP'IN KENDİSİ haberdar olma demektir.

Async Alternatif: Command/Reply Pattern

Production sistemlerinde genellikle REST yerine kuyruk (message broker) kullanılır:

- Orchestrator **PaymentCommand** kuyruğuna mesaj koyar
- Payment Service **PaymentCommand**'u dinler, işini yapar
- Payment Service **PaymentReply** kuyruğuna sonucu yazar
- Orchestrator **PaymentReply**'i dinler, sonucu alır, devam eder

Burada orchestrator dinliyor ama çok dar bir şey dinliyor: **kendi gönderdiği command'ın reply'ı**. Choreography'deki gibi 50 farklı event dinlemek zorunda değil.

Servisler ise sadece kendi command'larını dinler — yine kim gönderdiğini bilmezler.

DI Prensiplerine Uygunluk

Sık sorulan endişe: "Orchestrator her servisi bildiğine göre, bu coupling değil mi? Choreography'den daha sıkı bağımlılık doğurmuyor mu?"

Aslında tam tersi. Bağımlılık yönüne bak:

- **Orchestrator** → Payment, Inventory, Shipping (üst seviye, akışı yöneten — bilen)
- **Payment, Inventory, Shipping** → kimseyi bilmez (alt seviye, kendi işini yapan)

Bu Dependency Inversion'ın (SOLID'in D'si) **tam olarak söylediği şey**: yüksek seviye modüller (akış mantığı) düşük seviye modüllere (iş mantığı) bağlanabilir, alt seviye üst seviyeyi bilmez. Payment Service'in tek bildiği şey kendi public API'sıdır. Bu API'yı kim çağırırsa çağırırsın — mobile, web, test, orchestrator — Payment için fark etmez. Payment, orchestrator'ın varlığından bile haberdar değildir.

Choreography'de ise her servis başka servislerin event'lerini bilmek zorunda olduğundan, mesh tarzı kaotik bir bağımlılık grafiği oluşur. Bu yüzden orchestration DI prensiplerine choreography'den DAHA UYGUNDUR.

Multi-Orchestrator Yaklaşımı

Bir başka yaygın yanlış anlama: "Tek bir devasa orchestrator olur, 50 servisi tek başına yönetir, kod yüz binlerce satıra ulaşır."

Bu yanlış. Gerçekte her business process için ayrı bir orchestrator vardır.

E-commerce sisteminde örnek dağılım:

Orchestrator	Sorumluluğu	Bildiği servisler
OrderOrchestrator	Sipariş yerleştirme	Payment, Inventory, Shipping, Notification (~5)
RefundOrchestrator	İade işlemi	Payment, Inventory, Customer (~3)
SubscriptionOrchestrator	Abonelik yaşam döngüsü	Payment, Billing, AccessControl (~3)
OnboardingOrchestrator	Yeni kullanıcı kayıt	User, Email, Provisioning, Analytics (~4)
ReturnOrchestrator	Ürün iadesi	Shipping, Inventory, Payment, Quality (~4)

50 servisli bir sistemde belki 8-12 farklı orchestrator olur. Her biri **yalnızca kendi saga'sındaki servisleri bilir**. OrderOrchestrator, OnboardingOrchestrator'ın varlığından bile haberdar değildir.

50 Servisli Sistemde Karşılaştırma

	Choreography	Orchestration (çoklu)
Bağımlılık dağılımı	Her servis ~8-10 başkasının event'ini bilir → ~500 bağımlılık, mesh	Her orchestrator ~5 servis bilir × 10 orchestrator → ~50 bağımlılık, tree
Servisler ne bilir?	Her biri dolu, 10 event handler taşır	Sıfır bağımlılık, sadece kendi API'sı
Şişme nerede?	Her yerde dağılmış	Orchestrator'larda toplu ama her biri dar scope
Anlama maliyeti	Akış kodda yok, event'lerden çıkarmak gerekir	Her saga'nın akışı tek bir dosyada

Hangi Controller'lar Orchestrator Çağırır?

Her controller orchestrator çağırır. Aslında çoğu çağırır. Tipik bir sistemde controller'ların yaklaşık %80'i basit, %20'si orchestrated olur.

Orchestrator çağıran (saga-değer iş akışları):

- POST /api/orders → OrderController → OrderOrchestrator
- POST /api/refunds → RefundController → RefundOrchestrator
- DELETE /api/subscriptions/{id} → SubscriptionController → SubscriptionOrchestrator

Orchestrator gerektirmeyen (tek servisli basit işler):

- GET /api/users/{id} → UserController → UserService
- GET /api/products → ProductController → ProductService
- PUT /api/users/me/password → UserController → UserService
- GET /api/orders (listele) → OrderController → OrderService

Orchestrator gerekme şartları (üçü birden):

- 1) Birden fazla servis etkilenecek
- 2) Bu servisler ayrı veritabanlarına sahip (tek ACID transaction yok)
- 3) Hepsinin tutarlı olması iş açısından kritik

Üçü birden yoksa orchestrator overkill. Üçü birden varsa saga zamanı.

6. Isolation Problemi ve Çözümleri

Saga'da ACID'in "I"sı Yok

Saga'nın en sinsi zayıflığı isolation eksikliğidir. ACID'de Isolation şu demek: bir transaction tamamlanana kadar başka transaction'lar onun yaptıklarını göremez. Saga'da bu yok — çünkü **her lokal step hemen commit oluyor**. Saga'nın 3. adımı çalışırken, 1. ve 2. adımın etkileri zaten DB'de görünür durumda.

Üç Klasik Anomali

Problem	Ne demek	Örnek
Lost Updates	İki saga aynı veriyi günceller, biri diğerini ezer	İki sipariş aynı stok satırına yazıyor
Dirty Reads	Başka bir işlem henüz compensate edilmemiş ama edilecek veriyi okur	Müşteri hesabında çekilmiş parayı görür, sonra refund olur
Fuzzy Reads	Aynı veriyi iki kez okuyan farklı değer görür	Rapor çekerken saga step'i araya girer

Çözüm Yaklaşımları — Caitie McCaffrey Countermeasures

Caitie McCaffrey ve Chris Richardson literatürde beş ana çözüm tanımlar:

1. Semantic Lock

Application seviyesinde lock. Veriye "PENDING" / "IN_PROGRESS" gibi bir status flag konur. Diğer işlemler bu flag'i görünce ya beklerler ya da geri dönerler. **En sık kullanılan çözüm**. Veritabanı lock'undan farklı — sadece domain seviyesinde bir konvansiyon.

2. Commutative Updates

İşlemleri sıralamadan bağımsız tasarlırsın. Örneğin bakiyeye "+100" ve "-50" hangi sırayla gelirse gelsin sonuç aynı. Sıralama sorunu olduğu yerlerde uygulanabilir.

3. Pessimistic View

Saga adımlarını yeniden sıralarsın, riskli (compensation maliyetli) adımları sona atarsın. Doğrulamalar başa, para çekme sona — böylece dirty read olasılığı azalır.

4. Reread / Version Check (Optimistic Locking)

Update yapmadan önce veriyi tekrar okuyup version'ını kontrol edersin. Değişmişse abort edersin. Klasik optimistic concurrency control mantığı.

5. By Value

İş riskine göre strateji seçersin. Düşük tutarlı işlemler saga ile, yüksek riskli olanlar farklı bir consistency stratejisiyle (manuel onay, distributed lock, manuel reconciliation vb.).

Mülakat cümle olarak: "Saga'da ACID'in isolation garantisi yoktur, çünkü her lokal transaction hemen commit olur. Bu lost update, dirty read ve fuzzy read anomalilerine yol açabilir. Uygulamada bunu semantic lock (status flag ile), optimistic locking (version kontrolü) veya saga adımlarını yeniden sıralayarak çözeriz."

7. Gerçek Hayatta Saga Kullanımı

Workflow Engine'ler — Production'da Tercih

Production'da custom orchestrator yazmaktan ziyade workflow engine'leri tercih edilir. Bu framework'lerde saga deklaratif olarak yazılır, state otomatik persist edilir, retry/timeout/dead letter management hazır gelir, monitoring ve görselleştirme dahili olur.

Popüler workflow engine'ler:

- **Temporal** — Uber çıkışlı, çok popüler. Java/Go/Python/TypeScript SDK'ları var
- **Camunda** — BPMN diyagramıyla iş akışı çizersin. Kurumsal dünyada yaygın (banka, sigorta)
- **AWS Step Functions** — AWS ekosistemi için, JSON ile state machine yazılır
- **Netflix Conductor** — Netflix çıkışlı, microservice orkestrasyonu için
- **Axon Framework** — Spring/Java ekosisteminde event sourcing ve saga birlikte

Bu framework'lerin sağladıkları:

- **State persistence:** orchestrator çökse bile recovery — kaldığı yerden devam
- **Retry mekanizmaları:** network sorunlarında otomatik retry
- **Timeout yönetimi:** servis cevap vermezse ne yapılacağı
- **Dead letter queue:** defalarca başarısız olan mesajların ayrılması
- **Saga visualization:** akışı diagram olarak görmek
- **Distributed tracing entegrasyonu:** hangi adımda kaldı
- **Versioning:** saga akışlarının versiyonlanması — schema evolution

Saga'nın Yaygın Olduğu Sektörler

- **E-commerce:** Sipariş yerleştirme, iade, kargo takibi
- **Fintech:** Para transferi, ödeme, iade, KYC süreçleri
- **Booking/Travel:** Uçuş + otel + araba kombo rezervasyon
- **Subscription/SaaS:** Abone alımı, plan değişikliği, iptal
- **Telekom:** Abonelik aktivasyonu (SIM + hat + paket + fatura)
- **Insurance:** Poliçe çıkarma (underwriting + ödeme + dökümantasyon)
- **HR/ERP:** Yeni personel alımı (İK + Bordro + AD + Mail + IT ekipman)

Saga Ne Zaman Gereksizdir? — Alternatif Pattern'ler

Saga her yerde değildir. Bazı durumlarda daha basit çözümler tercih edilir:

Outbox Pattern

Database transaction'ı içinde "yapılacak iş" tablosuna kayıt at, ayrı bir job onu okuyup event'i yayınlasın. Atomicity garantili, saga'dan basit. Genelde async event publishing için kullanılır.

Eventual Consistency + Reconciliation Jobs

Async event'lerle ilerle, gece çalışan reconciliation job'larıyla kaybolan veya inconsistent kayıtları düzelt. Bazen saga'dan daha pragmatiktir, özellikle hata oranı düşükse.

Manual Approval Queues

Otomatik compensate edilemeyen durumlar için "operasyon ekibine düş" tarzı insan-bekleyen kuyruklar. Yüksek tutarlı, kritik işlemlerde tercih edilir.

Lokal Transaction

İş akışı tek bir servisi etkiliyorsa, klasik ACID transaction yeterlidir. Saga kullanmak overkill ve gereksiz karmaşıklık katar.

8. Mülakat Hazırlık

Tipik Soru Akışı

Mülakatçılar Saga konusunda genelde şu sırayla ilerler. Her aşamada sağlam cevap verirken bir sonraki seviyeye geçer; takılırsa o noktada kazır.

- 1. **"Saga pattern nedir?"** → Tanım
- 2. **"Neden gerekli?"** → Database-per-service + 2PC neden olmaz
- 3. **"Compensating transaction tam olarak nedir?"** → Detay, rollback değil
- 4. **"Saga'nın iki türü ve farkları?"** → Orchestration vs choreography
- 5. **"Hangisini ne zaman seçersin?"** → Karar matrisi, distributed monolith uyarısı
- 6. **"Saga'nın dezavantajları?"** → Isolation problemi, debugging zorluğu
- 7. **"Idempotency neden önemli?"** → Retry senaryoları
- 8. **"Real-world'de nasıl uygulanır?"** → Workflow engine'ler

Anahtar Cevap Kalıpları

Tanım cevabı

Saga pattern, mikroservis mimarilerinde birden fazla servisi kapsayan iş süreçlerinin tutarlılığını sağlamak için kullanılan bir patterndir. Klasik ACID transaction'ları kullanamadığımız durumlarda bir iş akışını birbirini takip eden lokal transaction'lar zinciri olarak modelleriz. Herhangi bir adım başarısız olursa compensating transaction'larla geri alırız. İki yaklaşımı vardır: orchestration ve choreography.

2PC neden değil cevabı

2PC her transaction için tüm servislerin kaynaklarını prepare phase'inde kilitler ve commit phase'ine kadar tutar. Bu kilitler throughput'u düşürür, tüm servislerin aynı anda ayakta olmasını gerektirdiği için başarısızlık olasılığı katlanır, coordinator single point of failure'dır. Saga kilit kullanmadan asenkron ilerlediği için yatay ölçeklenmeye uygundur.

Compensating transaction cevabı

Compensating transaction, zaten commit edilmiş bir lokal işlemin etkisini mantıksal olarak tersine çeviren YENİ bir transaction'dır. Database rollback DEĞİLDİR. Örneğin ödeme alındıktan sonra başka bir adım fail olursa, ödemeyi silmek yerine bir refund işlemi tetikleriz. Bu işlemlerin idempotent olması ve sonunda mutlaka başarılı olması kritiktir.

Orchestration vs choreography cevabı

Orchestration'da merkezi bir orchestrator akışı yönetir ve compensating transaction'ları o tetikler. Choreography'de servisler event'ler aracılığıyla

birbirleriyle haberleşir, merkezi kontrol yoktur. Orchestration debug ve visibility açısından rahat, complex compensation'da kolay. Choreography daha gevşek bağlıdır ama karmaşık akışlarda distributed monolith riski vardır.

Production kullanım cevabı

Saga her mikroservis sisteminde değil, birden fazla servisi etkileyen, eventual consistency'nin yeterli olmadığı ve compensating transaction gerektiren kritik iş akışları için kullanılır. Gerçek projelerde custom orchestrator yazmaktan ziyade Temporal, Camunda gibi workflow engine'leri tercih edilir; state persistence, retry, monitoring gibi cross-cutting concern'leri bu framework'ler çözer.

Hızlı Cheat Sheet

Soru	Tek Cümle Cevap
Saga nedir?	Distributed transaction'ları lokal transaction zinciri + compensating action ile yöneten pattern
Neden gerekli?	Database-per-service'te 2PC ölçeklenmez, global ACID kuramayız
Database-per-service ne?	Her servisin kendi DB'si, başkası direkt erişemez
Compensating = rollback mı?	Hayır — mantıksal "tersini yap", yeni bir forward operation
Hangi consistency modeli?	Eventual consistency (BASE)
Orchestration / Choreography?	Merkezi koordinatör vs event-driven
Choreography riski?	Distributed monolith — servisler birbirinin event'leriyle dolar
2PC neden olmaz?	Kilitler throughput'u öldürür, failure olasılığı katlanır, coordinator SPOF
Saga zayıflığı?	Isolation eksikliği — semantic lock ile aşılr
Compensating'in kritik özelliği?	Idempotent ve eventually successful olması
Saga state'i nerede?	Saga log (orchestrator'ın DB'sinde) — recovery için kritik
Orchestrator şişer mi 50 servis için?	Hayır — her business process için ayrı orchestrator vardır
Orchestrator failure'ı nasıl öğrenir?	Çağırdığı servisten gelen HTTP cevabıyla — event değil, request/response

9. Kapanış

Üç Cümlede Özet

- Saga, dağıtık sistemlerde gerçek rollback olmadan tutarlılık sağlama sanatıdır.
- Compensating transaction, rollback değil — semantic ters çevirmedi.
- Production'da kimse Saga'yı sıfırdan yazmaz — Temporal, Camunda gibi workflow engine'leri kullanır.

Akılda Kalsın Diye Hook'lar

Konsept	Hook
Saga'nın özü	ACID'in "C"sini koruyup "A" ve "I"dan vazgeçmek
Compensating transaction	Yumurta kırıldı, geri yapamazsın ama yeni yumurta alabilirsin
Orchestration	Orkestra şefi — manager Ali'ye "yap" der, cevabını bekler, Veli'ye "yap" der
Choreography	Dans grubu — şef yok, herkes müziği (event'i) dinliyor
2PC sorunu	Herkesi yemekte bekletir, şef haber vermezse salon kilitli kalır
Saga'nın ruhu	"Önce yap, sorun olursa düzelt" — eventual consistency mantığı
Compensating'in özelliği	Idempotent olmak şart — defalarca tetiklense de aynı sonuç
Distributed monolith	Görünüşte bağımsız, gerçekte event'lerle zincirli

Son Söz

Mülakatlarda Saga konusunun değerlendirilmesinde en çok ödüllendirilen şey: konsepti ezbere değil, **gerçek senaryolarla anlatabilmektir**. "Choreography distributed monolith'e dönüşebilir" gibi nuanced ifadeler, sadece tanımı verebilenden çok daha fazla derinlik gösterir.

Saga modern mikroservis mimarilerinin getirdiği consistency problemine zarif bir çözümdür. ACID'in iki harfinden — Atomicity ve Isolation — feragat ederek ölçeklenebilirlik ve fault-tolerance kazandırır. Bunun karşılığında eventual consistency ile yaşamayı, compensating transaction'lar tasarlamayı ve isolation eksikliğini yönetmeyi öğrenmek gerekir.

Bu doküman üzerinden 1-2 kez geçtikten sonra konsept yerleşecek. Hook'ları ve cheat sheet'i son 30 dakikada gözden geçirmek yararlı olur. Başarılar.

— Son —